

A CORRECTNESS PROOF

This section provides the full proofs of Theorems 1, 2, and 3.

A.1 Safety

We assume a Byzantine fault model with $n = 3f+1$ replicas, where at most f replicas are Byzantine. A *strong quorum* requires $Q = 2f+1$ votes, and a *weak quorum* requires $Q_{\text{weak}} = f+1$ votes. We say two proposals *conflict* if neither is an ancestor of the other.

LEMMA 1 (QUORUM INTERSECTION). *Any two strong quorums of size $2f+1$ intersect in at least $f+1$ replicas; in particular, they share at least one correct replica.*

PROOF. Let Q_1 and Q_2 be two strong quorums with $|Q_1| = |Q_2| = 2f+1$. Since $n = 3f+1$,

$$|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - n = (2f+1) + (2f+1) - (3f+1) = f+1.$$

As at most f replicas are Byzantine, $Q_1 \cap Q_2$ contains at least one correct replica. $\square$

LEMMA 2 (NO DOUBLE VOTE). *A correct replica votes for at most one proposal in a given round.*

PROOF. Correct replicas follow the protocol. In each round r , a correct replica selects a single candidate proposal according to the priority rule and broadcasts VOTE messages for exactly that proposal. $\square$

LEMMA 3 (PoR UNIQUENESS). *For any round r , there cannot exist two valid PoR certificates that certify conflicting proposals.*

PROOF. Assume for contradiction that there exist two valid PoRs in round r certifying conflicting proposals b and b' . Each PoR requires $2f+1$ votes. By Lemma 1, the two voter sets intersect in at least one correct replica. That replica would have voted for both b and b' in round r , contradicting Lemma 2. $\square$

LEMMA 4 (LOCK MONOTONICITY). *For any correct replica p_i , its lock $lock_i$ is monotonically non-decreasing in round number. Moreover, once p_i observes a PoR for a proposal, it locks that proposal and will not vote for any conflicting proposal unless it later observes a strictly higher-round PoR on the competing proposal's certified ancestry.*

PROOF. Whenever p_i observes a valid PoR_r for proposal b_r , it updates $lock_i$ to b_r unless it is already locked on a descendant of b_r . Thus $\text{round}(lock_i)$ can only increase over time.

After locking, SAFETOVOTE allows p_i to vote only for a proposal that either (i) extends the current locked branch, or (ii) carries a strictly higher-round PoR that justifies switching. By protocol validity, any such higher-round PoR must lie on the proposal's certified ancestry. Therefore, p_i never votes for a conflicting proposal unless it has first observed higher certified evidence that justifies the switch. $\square$

LEMMA 5 (COMMIT IMPLIES A LOCKED SET). *If a proposal $w = b_r$ is committed by the Two-PoR commit rule via a certified chain $b_r \leftarrow b_{r+1}$ with PoRs PoR_r and PoR_{r+1} , then there exists a set L of at least $f+1$ correct replicas such that every replica in L is locked on w (or its descendants).*

PROOF. Let Q_{r+1} be the voter set that formed PoR_{r+1} for b_{r+1} . Since $|Q_{r+1}| = 2f+1$ and at most f replicas are Byzantine, Q_{r+1} contains at least $f+1$ correct replicas; let L be those correct voters.

By protocol validity, a correct replica votes for b_{r+1} only after verifying that it extends b_r and that its certified justification includes the parent certificate PoR_r on b_r (carried either directly as the round-entry certificate or as the highest justified PoR on b_{r+1} 's certified ancestry). Hence, every replica in L has observed PoR_r before voting for b_{r+1} . By Lemma 4, each such replica locks on $w = b_r$ (or a descendant on the same branch). $\square$

Theorem 1 (Safety). *No two correct replicas commit (finalize) conflicting proposals.*

PROOF OF THEOREM 1. Assume for contradiction that two correct replicas commit conflicting proposals w and b , with $\text{round}(w) < \text{round}(b)$.

Since w is committed, by Lemma 5, there exists a set L of at least $f+1$ correct replicas locked on w (or its descendants).

Since b is committed, there must exist at least one PoR that certifies a proposal conflicting with w in some round after $\text{round}(w)$. Let PoR^* be the earliest such PoR, certifying a proposal x^* that conflicts with w , and let Q^* be its voter set.

Because $|Q^*| = 2f+1$ and $|L| = f+1$, we have

$$|Q^* \cap L| \geq (2f+1) + (f+1) - (3f+1) = 1.$$

Thus, Q^* intersects L in at least one correct replica p .

Replica p is locked on the branch of w . For p to vote for x^* , SAFETOVOTE requires that x^* either extends p 's locked branch or carries a strictly higher-round PoR on x^* 's certified ancestry that justifies switching. However, x^* conflicts with w , so it does not extend the locked branch. Moreover, any such higher-round PoR on x^* 's ancestry would itself certify a conflicting proposal earlier than PoR^* , contradicting the choice of PoR^* as the earliest conflicting PoR after w . Therefore, p cannot vote for x^* , contradicting $p \in Q^*$.

Hence, no two correct replicas can commit conflicting proposals. $\square$

A.2 Liveness

Definition 1 (Timeout-Sufficient Replica). *Fix a stabilized synchronous interval whose actual message delay bound is Δ . A correct replica p_i is timeout-sufficient during that interval if its local timeout baseline satisfies $\delta_i \geq \Delta$.*

LEMMA 6 (EVENTUAL TIMEOUT SUFFICIENCY). *During any sufficiently long stabilized synchronous interval in which at least $2f+1$ correct replicas are mutually connected, every correct replica eventually becomes timeout-sufficient.*

PROOF. Fix such a stabilized synchronous interval with actual delay bound Δ . Consider any correct replica p_i .

If p_i cannot yet collect enough $\langle \text{SYNC-READY}, v \rangle$ messages to locally form a synchronization certificate, it keeps retransmitting its $\langle \text{SYNC-READY}, v \rangle$ message. Since at least $2f+1$ correct replicas are mutually connected and all messages among them are delivered within Δ , eventually every correct replica receives $2f+1$ valid SYNCREADY messages for some synchronization view and forms a local $\langle \text{SYNC-CERT}, v \rangle$.

If replica p_i locally forms a synchronization certificate but does not receive certificates from at least $f+1$ distinct replicas within time δ_i , it doubles its δ_i and retries in a higher synchronization view. Hence, as long as $\delta_i < \Delta$, repeated failures cause δ_i to grow exponentially.

Once $\delta_i \geq \Delta$, after p_i forms its local synchronization certificate, every correct replica that is already in the same synchronization view can form and broadcast its own synchronization certificate within at most Δ , and that certificate is delivered back to p_i within at most another Δ . Therefore, replica p_i receives remote synchronization certificates from at least $f+1$ distinct replicas within time δ_i , so the calibration attempt succeeds.

Thus, after finitely many retries, every correct replica reaches a timeout baseline $\delta_i \geq \Delta$ and remains timeout-sufficient throughout the stabilized interval. $\square$

Definition 2 (Good round). *A round r is good if there exists a stabilized synchronous interval of length $\geq 3\Delta$, partitioned into three consecutive subintervals I_1 , I_2 , and I_3 each of length $\geq \Delta$, such that: (i) by the end of I_1 , all correct replicas have entered round r and are timeout-sufficient; (ii) during $I_1 \cup I_2$, every proposal broadcast by a correct replica in round r is delivered to every correct replica; and (iii) during I_3 , all correct replicas remain in round r , and every vote broadcast by a correct replica in round r is delivered to every correct replica.*

LEMMA 7 (GOOD ROUND EXISTS). *During any sufficiently long stabilized synchronous period, the pacemaker ensures that all correct replicas eventually synchronize their round counters and enter good rounds.*

PROOF. Consider a stabilized synchronous period in which messages between correct replicas are delivered within Δ . By Lemma 6, after finitely many retries, every correct replica becomes timeout-sufficient. If any correct replica enters a higher round using a valid PoR or RC, it broadcasts that certificate. If a lagging correct replica has not yet seen the standalone certificate, it will still receive a valid higher-round proposal carrying the same certificate as its *enterCert*. In either case, within at most one message delay, every correct replica receives round-entry evidence for that higher round and can catch up.

Moreover, when a lagging replica catches up to an already active round, it joins that round using only the remaining local waiting needed before entering the election phase, rather than restarting a full fresh round. Thus, during a sufficiently long stabilized synchronous period, the skew among correct replicas is continuously reduced, and there exists a round r in which all correct replicas enter early enough to complete the proposal-exchange window ($> 2\Delta$) and the election window ($> \Delta$) together. Hence, round r is good by Definition 2.

As the stabilized synchronous period continues, the same argument applies repeatedly, so good rounds recur. $\square$

LEMMA 8 (PoR IN A GOOD ROUND (PROBABILISTIC)). *In any good round r , honest replicas form a valid PoR for a single proposal with probability at least $\frac{2f+1}{n}$.*

PROOF. Fix a good round r . By Definition 2, by the end of the proposal-exchange window ($I_1 \cup I_2$), every correct replica has received every proposal broadcast by a correct replica in round r . We assume that each replica accepts at most one proposal per author per round, ignoring additional proposals from the same author as equivocations. Hence, the number of candidate proposals considered in round r is at most n .

Let $\mathcal{M}_r$ denote the set of proposals with maximum log strength under $\mathcal{S}(\cdot)$ among the proposals received by a correct replica in round r . If $|\mathcal{M}_r| = 1$, all correct replicas select the same strongest proposal and vote for it.

Otherwise, ties are broken using the common coin C_r for round r . Since C_r is obtained by threshold aggregation of $f+1$ signature shares from $f+1$ distinct replicas, all correct replicas compute the same C_r . For each proposal $\mathcal{P} \in \mathcal{M}_r$, replicas compute

$$\text{Score}(\mathcal{P}) = H(C_r \parallel \mathcal{P}.\text{proposer_id})$$

and select the proposal with the highest score.

Consider the event that the highest score among the tied proposers corresponds to an honest proposer. Since proposer identities are fixed and C_r is unpredictable to the adversary before round r , the top-scoring proposer is uniformly distributed over the n proposers, and is honest with probability $\frac{2f+1}{n}$. Conditioned on this event, all correct replicas select the same honest proposal.

During the election window I_3 , all correct replicas broadcast their VOTES, and by Definition 2, those votes are delivered to every correct replica within one message delay. Therefore, the selected proposal receives at least $2f+1$ votes, allowing correct replicas to assemble a valid PoR in round r . $\square$

Theorem 2 (Liveness (with probability 1)). *During any sufficiently long stabilized synchronous period, correct replicas commit new proposals continuously; over an unbounded sequence of such stabilized periods, commits occur infinitely often with probability 1.*

PROOF OF THEOREM 2. Consider a sufficiently long stabilized synchronous period. By Lemma 6, sufficiently long stabilized synchronous periods first make all correct replicas timeout-sufficient; then, by Lemma 7, good rounds recur. In each good round, Lemma 8 implies that a PoR is formed with non-zero probability.

Assuming fresh common-coin outputs across rounds, these per-round success events occur with a fixed non-zero probability, so over an unbounded sequence of good rounds, PoRs occur infinitely often with probability 1.

Whenever two consecutive rounds produce PoRs for proposals that extend one another, the Two-PoR commit rule commits the earlier proposal in the two-chain. Hence, during any sufficiently long stabilized synchronous period, correct replicas commit new proposals continuously; over an unbounded sequence of such stabilized periods, commits occur infinitely often with probability 1. $\square$

A.3 Partial Liveness

CASSANDRA distinguishes between *commitment* (which requires strong quorums and PoRs) and *partial progress* (which preserves useful work and availability evidence during partitions).

LEMMA 9 (WEAK QUORUM PROGRESS). *In any connected component that contains at least $Q_{\text{weak}} = f+1$ honest replicas, whose*

internal message delays are bounded by Δ , and in which all participating correct replicas are timeout-sufficient (i.e., they have local timeout baselines satisfying $\delta_i \geq \Delta$), the component can form at least one PoA in a round.

PROOF. Let C be a connected component containing $k \geq f+1$ honest and timeout-sufficient replicas. Within C , all honest replicas receive each other's proposals within the proposal-exchange window ($> 2\Delta$). By applying the deterministic *Priority Rule*, these replicas converge on the same strongest candidate and broadcast votes for it during the subsequent election window ($> \Delta$). Since $|C| \geq Q_{\text{weak}} = f+1$, replicas are guaranteed to aggregate the required weak quorum of votes. Consequently, replicas in C can locally assemble a valid PoA. $\square$

LEMMA 10 (RC PROGRESS IN A WEAK COMPONENT). *In any connected component that contains at least $Q_{\text{weak}} = f+1$ honest replicas, the component can form a valid RC for round $r + 1$.*

PROOF. If no PoR is formed in round r , then every correct replica in the component eventually reaches the election timeout for round r and broadcasts a WISHPROPOSAL(r) message. Since the component contains at least $f+1$ honest replicas and those messages are eventually delivered (without strict timing guarantees), replicas in the component can collect at least $f+1$ such messages. By the protocol rule for round synchronization, these messages form a valid RC for round $r + 1$. $\square$

LEMMA 11 (DATA AVAILABILITY). *If a proposal is certified by a PoA or a PoR, its data is available at at least one correct replica. Hence, any correct replica can lazily retrieve it via the catch-up mechanism when connectivity permits.*

PROOF. A PoA or a PoR requires at least $f+1$ votes, which guarantees that at least one correct replica has verified and stored the proposal. Hence, this certificate ensures that the proposal remains retrievable in the system. $\square$

Theorem 3 (Partial Liveness). *CASSANDRA satisfies the **Partial Liveness** property. Specifically, during network partitions, any connected component that contains at least $f+1$ correct replicas and whose internal message delays are bounded by Δ , and whose correct replicas use local timeout baselines satisfying $\delta_i \geq \Delta$, continues to advance rounds locally and produce PoA-certified partial progress. After network recovery, this accumulated partial progress remains recoverable and can be incorporated into subsequent PoR-backed commitment.*

PROOF OF THEOREM 3. Let C be a connected component containing at least $f+1$ correct and timeout-sufficient replicas and timely internal communication.

By Lemma 9, in each local round, replicas in C can form a PoA for the strongest locally visible proposal, thereby preserving certified partial progress. If the component cannot form a strong certificate in that round, then by Lemma 10, the same replicas can still form an RC for the next round and continue advancing locally. Thus, even without a global strong quorum, C continues both to make round progress and to accumulate recoverable PoA-certified work.

A PoA does not imply commitment, but by Lemma 11, it guarantees that the certified proposal remains available at some correct replica. Once global connectivity is restored and a strong quorum

$Q = 2f+1$ becomes reachable again, replicas exchange their local histories, compare the available branches via the deterministic priority rule, and resume forming PoRs on the strongest recoverable branch. The catch-up and back-fill mechanisms then retrieve and extend previously accumulated PoA-backed proposals on that branch, allowing the system to reincorporate partition-time partial progress into subsequent PoR-backed commitment.

Therefore, CASSANDRA satisfies Partial Liveness. $\square$

B RELATED WORK

The design of CASSANDRA draws inspiration from and builds upon several decades of research in BFT consensus. We categorize the most relevant literature into four primary domains: partition-tolerant consensus, sharding protocols, asynchronous BFT, and view synchronization.

Network Partition. The challenge of maintaining consistency and availability under network partitions is fundamentally tied to the CAP theorem. Traditional BFT protocols typically prioritize safety and stall indefinitely when a strong quorum ($2f+1$) cannot be assembled. *Partitionable Blockchain Consensus* [26] was among the first to formalize this problem. It proves impossibility in a fully asynchronous model, then leverages external *partition detectors* to coordinate the partition. Raptr [49] introduces prefix consensus: replicas maintain safety by finalizing only the common transaction prefix. Then it reconciles divergent suffixes upon restoration. More recently, AutoBahn [19] preserves a limited form of partial progress under network disruptions, but this progress is largely confined to the dissemination layer rather than the ordering layer, and thus does not directly address *designated-leader dependence*.

In contrast to these approaches, CASSANDRA neither relies on external partition detectors nor confines partial progress to data dissemination alone. Instead, its core ordering path allows partitions to independently order transactions that are later integrated through an implicit merge method.

View Synchronization. View synchronization is fundamental for aligning replicas within a common round to ensure sustained consensus progress. While classic protocols like PBFT [9] utilize an exponential back-off strategy for eventual convergence, HotStuff [52] decouples this logic into a dedicated *pacemaker*. Building on this modularity, state-of-the-art mechanisms have evolved to address the quadratic communication overhead inherent in traditional designs. Notable advancements include Lewis-Pye's hierarchical, epoch-based framework [35, 36], which achieves amortized $O(n)$ communication complexity, and Lumiere [37], which further optimizes steady-state performance with optimistic responsiveness.

While these advancements significantly enhance steady-state performance, they remain fundamentally dependent on a $2f+1$ strong quorum of "new-view" messages to transition between rounds, leading to total system stalls during network partitions. Cassandra fills this operational void by introducing a novel pacemaker that facilitates round advancement through the evidence from $f+1$ replicas, thereby sustaining partial progress where other state-of-the-art pacemakers fail to coordinate.

Dual-Path Design BFT. A common theme in practical BFT systems is to expose an *optimistic fast path* that achieves low latency and

linear communication when the network is stable and the active leader is responsive, while retaining a *fallback path* that preserves safety under faults and delays.

Prior dual-path BFT protocols [20, 34, 42] achieve low latency via an optimistic fast path but typically remain leader-driven in the common case and rely on leader-based recovery to fall back under adverse conditions. DAG-based designs such as Bullshark [46] similarly incorporate a predefined leader in the synchronous path, while relying on a random coin to select a fallback leader for global convergence. In contrast, although CASSANDRA also employs a coin, it is used only as a tie-breaker when multiple proposals remain indistinguishable under the deterministic priority rule, rather than as a mechanism for leader election or round synchronization. As a result, CASSANDRA removes designated-leader (and coin-driven leader-election) dependence from its core ordering path via partitionable leader selection and deterministic proposal priority, enabling components to continue making order-relevant partial progress even when global leader reachability or coin-driven coordination becomes ineffective under partitions.

Sharding Protocols. Sharded systems aim to improve scalability by partitioning replicas into smaller committees that process disjoint subsets of state or transactions in parallel. Early systems such as Chainspace [2] explored object-centric sharding with BFT committees and atomic object locking. Building on this, OmniLedger [33] utilized secure randomness for epoch-based committee assignment, while RapidChain [53] pipelines block production across shards. Cerberus [24] further evolved this by employing per-shard DAGs to eliminate single-leader bottlenecks. Extending this line of work, ByShard [25] optimizes this paradigm by introducing a sharded database framework that efficiently integrates BFT consensus with atomic cross-shard coordination. RCanopus [32] geographically divides replicas into Byzantine Groups (BGs) that run local BFT, while a super-consensus orders the BG outputs.

While these systems significantly improve horizontal scalability, they still inherently stall if fewer than $2f+1$ replicas remain in a shard after a partition. CASSANDRA's partial-progress technique is *orthogonal*. It could serve as an intra-shard consensus component, allowing each committee to process local transactions with non-zero throughput even when a partition leaves only $f+1$ honest replicas connected.

Concurrent and Multi-Leader BFT. Recent work [7, 22, 29, 38–40, 47, 48] has also explored allowing multiple replicas to concurrently drive consensus, which helps reduce the bottleneck of a single designated leader under stable conditions. At a high level, this design may appear to mitigate *leader dependence*, since progress is no longer tied to one pre-assigned proposer. However, these protocols still require global coordination to reconcile concurrent proposals and remain constrained by the *strong quorum dependence*. While they improve steady-state throughput, they do not directly address the partition setting considered in CASSANDRA: failing to make useful progress during network partitions.

Asynchronous Protocols. Fully asynchronous protocols, such as HoneyBadgerBFT [43] and BEAT [15], eliminate timing assumptions by combining reliable broadcast with threshold cryptography. Recent advancements in DAG-based BFT, including DAG-Rider [31]

and Tusk [14], further optimize performance by decoupling data dissemination from ordering, achieving optimal resilience and amortized $O(n^2)$ complexity. Fides [51] further enhances this model by offloading critical operations into trusted hardware for an optimized threshold and better performance.

Despite their resilience to asynchrony, these protocols remain bound by the *Strong Quorum Dependence*: none can make progress once fewer than a quorum of replicas are reachable, since they still require a classical quorum to form certificates. In contrast, CASSANDRA addresses this by allowing a weaker quorum to construct weaker certificates, achieving non-zero progress during weak quorums—a breakthrough from 0 to 1.